

L4-P08

低比特微调与量化

实验手册

涵盖 QAT 感知量化训练 · QLoRA 微调 · 层重要性分析 · GGUF 格式制作
平台：MoPaaS | 框架：PyTorch 2.4 | 模型：Qwen1.5-1.8B-Chat

实验编号	实验名称	核心技术	关键依赖
Ex 1	QAT 感知量化训练	FakeQuant / STE / QLinear	torch, transformers
Ex 2	QLoRA 微调	4bit NF4 + LoRA Adapter	bitsandbytes, peft, accelerate
Ex 3	层重要性分析	噪声扰动 / Δ Loss 排序	torch, transformers, tqdm
Ex 4	GGUF 格式制作	llama.cpp / Q4_K_M / Q8_0	llama.cpp, Ollama

实验 1: QAT 感知量化训练

程序文件: ex1_qat_train.py

实验目标: 对 FP16 模型执行感知训练模拟量化 (Quantization-Aware Training)。训练期间通过伪量化算子在 forward 阶段模拟整数化误差, 并使用直通估计器 (STE) 保证反向传播梯度流通。

1.1 环境依赖

安装所需依赖 (推荐指定版本以保证复现性):

```
# 通用安装
pip install torch transformers

# 指定版本 (推荐)
pip install torch==2.1.0 transformers==4.49.0
```

 **提示:** 在 MoPaaS 平台创建开发环境时, 请选择 PyTorch 2.4 镜像, 已预装上述依赖。

1.2 程序核心模块

本实验由以下 4 个核心模块组成:

① 自定义伪量化函数 FakeQuantSTE

基于 PyTorch autograd 自定义算子, 实现前向量化 + 反向直通梯度 (STE)。

```
class FakeQuantSTE(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, scale):
        q = torch.clamp(torch.round(x / scale), -127, 127)
        return q * scale    # 伪量化 + 反量化

    @staticmethod
    def backward(ctx, grad_output):
        return grad_output, None    # 直通梯度 (STE)
```

前向传播将浮点张量量化到 INT8 范围并反量化, 反向传播梯度直接透传, 不因量化操作截断。

② 量化线性层 QLinear

替换标准 nn.Linear, 在 forward 阶段对权重逐通道量化、对激活逐张量量化。

```
class QLinear(nn.Module):
    def forward(self, x):
        w_scale = calc_channelwise_scale(self.weight, per_channel=True)
        w_q      = FakeQuantSTE.apply(self.weight, w_scale)
        if self.quantize_activation:
            a_scale = calc_channelwise_scale(x, per_channel=False)
```

```
x = FakeQuantSTE.apply(x, a_scale)
return F.linear(x, w_q, self.bias)
```

③ 递归替换 `Linear` → `QLinear`

遍历模型所有子模块，将普通 `nn.Linear` 替换为 `QLinear`，同时跳过 `embedding` / `layernorm` 等敏感层。

```
def replace_linear_with_qlinear(module):
    for name, child in module.named_children():
        if isinstance(child, nn.Linear) and not is_sensitive_module_name(name):
            setattr(module, name, QLinear(...))
        else:
            replace_linear_with_qlinear(child)
```

④ 训练与运行

```
python ex1_qat_train.py \
    --model_path    "../models/Qwen1.5-1.8B-Chat" \
    --output_dir    "../models/Qwen1.5-1.8B-Chat-QAT-demo" \
    --epochs 1 --lr 2e-5 --batch_size 1
```

1.3 预期输出

```
📦 模型参数近似内存: 3.52 GB (dtype=fp16)
🔑 正在注入 QAT 量化层 ...
🚀 开始训练 ...
✅ 训练完成
💾 保存 QAT 模型到 ../models/.../QAT-demo
```

实验 2：QLoRA 微调

程序文件：ex2_qlora_finetune.py

实验目标：在显存有限（单卡 16 GB）的环境下，使用 QLoRA 对大语言模型进行高效微调。QLoRA 结合 4-bit NF4 量化（极低显存占用）与 LoRA 低秩适配（仅训练少量增量参数），是目前最主流的轻量化微调方案。

 **注意：**当前 MoPaaS 平台暂不支持原生 4-bit 量化，本实验使用兼容模式（FP16）代替，代码结构与真实 QLoRA 完全一致。

2.1 环境依赖

```
# 通用安装
pip install bitsandbytes peft transformers accelerate

# 指定版本（推荐）
pip install accelerate==1.4.0 bitsandbytes==0.48.1 \
    peft==0.15.0 transformers==4.50.0
```

2.2 核心原理

QLoRA 的数学表达如下：

$$\mathbf{W}' = \mathbf{W} + \mathbf{A} \cdot \mathbf{B}$$

其中 $\mathbf{W}$ 为原模型 4-bit 冻结权重， $\mathbf{A}$ 、 $\mathbf{B}$ 为可训练低秩矩阵（float16 精度）。

模块	技术	作用
BitsAndBytes	4-bit NF4 量化	压缩模型权重，大幅节省显存占用
LoRA	低秩矩阵分解	仅训练少量增量参数（< 0.2%）
PEFT	参数高效微调框架	自动注入 LoRA 层并管理保存
HF Trainer	Hugging Face 训练框架	自动化梯度计算、日志、GPU 分配

2.3 关键配置参数

LoRA 配置

```
LoraConfig(
    r=8,                                # 低秩维度
    lora_alpha=16,                       # 缩放因子
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
)
```

量化配置（BitsAndBytes）

```
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # 正态浮点量化，优于 INT4
    bnb_4bit_compute_dtype=torch.bfloat16,
)
```

2.4 训练流程

`train_qlora()` 函数按以下步骤执行：

1. 固定随机种子（`set_seed`）
2. 加载模型与分词器
3. 注入 LoRA 层（`get_peft_model`）— 可训练参数约 0.17%
4. 构建 Toy 数据集并进行 `tokenize`（用户-助手格式拼接）
5. 配置并启动 HF Trainer
6. 推理演示（验证微调效果）
7. 保存 LoRA adapter 权重与元信息 JSON

2.5 运行命令

基础示例（最小配置）：

```
python ex2_qlora_finetune.py \
    --model_name_or_path ../models/Qwen1.5-1.8B-Chat \
    --output_dir ../qlora-output \
    --epochs 1 --batch_size 1 --lr 2e-4 --max_length 512
```

2.6 命令行参数说明

参数	说明
<code>--model_name_or_path</code>	模型路径（Hugging Face 格式目录）
<code>--output_dir</code>	微调结果保存目录
<code>--epochs</code>	训练轮数，建议 1~3
<code>--batch_size</code>	每卡每次迭代的样本数
<code>--lr</code>	学习率（常用 $1e-4 \sim 5e-4$ ）
<code>--max_length</code>	最大输入序列长度（tokens）

2.7 输出文件结构

```
qlora-output/  
├─ adapter_model.bin      # LoRA 权重  
├─ adapter_config.json    # LoRA 配置  
├─ tokenizer.json         # 分词器  
└─ training_meta.json     # 训练元信息
```

实验 3：计算权重重要性（层敏感性分析）

程序文件：ex3_layer_importance.py

实验目标：通过对各 **Linear** 层施加微小噪声扰动，量化其对模型输出损失的影响（ ΔLoss ），从而识别对量化最敏感的层，并生成 **FP16** 白名单供后续量化工具使用。

3.1 环境依赖

```
pip install torch transformers tqdm

# 验证版本
pip list | grep -E "torch|transformers|tqdm"
# torch 2.4.1 | transformers 4.45.0 | tqdm 4.67.1
```

3.2 算法流程

程序按以下 5 步执行：

1. 加载模型与分词器，设置 **eval** 模式
2. 计算基线 **Loss**（对若干示例文本求平均 **Loss**）
3. 逐层敏感性分析：对每个 **Linear** 层注入微小高斯噪声，重新前向计算，记录 ΔLoss
4. 结果保存：将 ΔLoss 排序后写入 **CSV** 文件
5. 生成白名单：选取 ΔLoss 最大的 10% 层（量化敏感层）写入 **JSON** 文件

3.3 运行命令

```
python ex3_layer_importance.py
```

3.4 输出文件说明

输出文件	内容说明
layer_importance_qwen15.csv	layer_name（层路径）+ delta_loss（ ΔLoss ），按重要性降序排列
keep_in_fp16_qwen15.json	需保留为 FP16 的层名数组，可直接用于量化工具 keep_in_fp16 参数

3.5 FP16 白名单的使用方法

生成的白名单可用于主流量化框架，保留关键层精度，避免量化导致模型退化（通常可避免 2%–5% 精度损失）。

① AutoAWQ (AWQ 量化)

```
import json
from autoawq import AutoAWQForCausalLM
from transformers import AutoTokenizer

with open("keep_in_fp16_qwen15.json") as f:
    keep_in_fp16 = json.load(f)

model = AutoAWQForCausalLM.from_pretrained(...)
model.quantize(
    tokenizer=tokenizer,
    quant_config={
        "w_bit": 4,
        "q_group_size": 128,
        "keep_in_fp16": keep_in_fp16,  # ☒ 指定敏感层保留 FP16
    }
)
model.save_quantized("../models/Qwen1.5-1.8B-Chat-AWQ-INT4")
```

② GPTQ 量化

```
# 代码方式
GPTQQuantizer(
    model=model, tokenizer=tokenizer,
    bits=4, group_size=128,
    keep_in_fp16="keep_in_fp16_qwen15.json",  # ☒
)

# 命令行方式
python gptq_quantize.py \
    --model ../models/Qwen1.5-1.8B-Chat \
    --keep_in_fp16 keep_in_fp16_qwen15.json \
    --bits 4 --group_size 128
```

③ SmoothQuant (Intel Neural Compressor, INT8)

```
import json
from optimum.intel.neural_compressor import INCQuantizer
from neural_compressor.config import PostTrainingQuantConfig

with open("keep_in_fp16_qwen15.json") as f:
    keep_in_fp16 = json.load(f)

quantizer = INCQuantizer.from_pretrained(model)
quant_config = PostTrainingQuantConfig(
    approach="smooth_quant",
```



```
    keep_in_fp16=keep_in_fp16,    # ☒
)
quantizer.quantize(save_directory="../models/Qwen1.5-SQ-INT8",
                   quant_config=quant_config)
```

实验 4：制作 GGUF 格式

实验目标：将 Hugging Face 格式的模型（safetensors / bin 权重）转换为 .gguf 文件，并进一步量化为 Q4_K_M、Q5_K_M 或 Q8_0 等紧凑格式，以便通过 llama.cpp / Ollama / LM Studio 等工具在本地高效运行。

4.1 整体流程

步骤	说明
① 准备 Hugging Face 模型目录	需包含 config.json、tokenizer 及完整权重文件
② 转换为 FP16 GGUF（中间格式）	使用 convert_hf_to_gguf.py
③ 量化 GGUF（目标格式）	使用 llama-quantize 工具生成 Q4/Q5/Q8
④ 测试运行 / 导入 Ollama	用 llama-cli 测试或通过 Modelfile 导入 Ollama

 **提示：**如果你的模型是 LoRA 微调结果，需先将 LoRA adapter 合并回基座模型，再执行 GGUF 转换。

4.2 安装 llama.cpp

课件已提供压缩包，解压后直接使用。如需从源码安装：

```
# 克隆仓库
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp

# 安装 Python 依赖
pip install -r requirements.txt

# 编译 (Linux / macOS)
cmake -B build
cmake --build build --config Release
```

4.3 准备模型目录

模型目录须为完整的 Hugging Face 格式，示例如下：

```
Qwen2.5-7B-Instruct/
├── config.json
├── tokenizer.json
├── tokenizer_config.json
├── model-00001-of-00004.safetensors
├── model-00002-of-00004.safetensors
├── model-00003-of-00004.safetensors
├── model-00004-of-00004.safetensors
└── model.safetensors.index.json
```

4.4 转换为 FP16 GGUF

```
cd /ai/work/P08/ex4_llama_cpp

python llama.cpp-master/convert_hf_to_gguf.py \
  /ai/work/models/Qwen1.5-1.8B-Chat \
  --outfile Qwen1.5-1.8B-Chat-f16.gguf \
  --outtype f16
```

生成 Qwen1.5-1.8B-Chat-f16.gguf 中间文件（文件较大，仅用于后续量化步骤）。

4.5 量化 GGUF

先配置环境变量，再执行量化：

```
cd /ai/work/P08/ex4_llama_cpp
export LD_LIBRARY_PATH=llama.cpp-master/build/bin:$LD_LIBRARY_PATH
export PATH=llama.cpp-master/build/bin:$PATH

# Q4_K_M — 最常用，体积小，速度快
llama-quantize Qwen1.5-1.8B-Chat-f16.gguf \
  Qwen1.5-1.8B-Chat-q4_k_m.gguf Q4_K_M

# Q5_K_M — 精度更好，文件稍大
llama-quantize Qwen1.5-1.8B-Chat-f16.gguf \
  Qwen1.5-1.8B-Chat-q5_k_m.gguf Q5_K_M

# Q8_0 — 接近原始精度，文件较大
llama-quantize Qwen1.5-1.8B-Chat-f16.gguf \
  Qwen1.5-1.8B-Chat-q8_0.gguf Q8_0
```

4.6 量化格式选择指南

格式	精度 / 体积特点	推荐场景	备注
F16	精度最高，文件最大	服务器推理 / 中间格式	—
Q8_0	精度较好，压缩较少	对质量要求高	—
Q5_K_M	精度与体积较为平衡	教学 / 应用部署	—
Q4_K_M ★	最常用，体积小，速度快	本地部署 / 普通 PC	首选
Q3_K_M	更小，质量有所下降	内存极度有限	—

格式	精度 / 体积特点	推荐场景	备注
Q2_K	极小，损失较大	不推荐正式使用	慎用

 **提示：** 一般优先使用 Q4_K_M；若显存 / 内存充足，可使用 Q5_K_M 或 Q8_0 以获得更好效果。

4.7 测试与部署

llama.cpp 命令行测试

```
llama-cli -m Qwen1.5-1.8B-Chat-q4_k_m.gguf \  
-p "请介绍一下人工智能"
```

启动本地 API 服务

```
llama-server -m Qwen1.5-1.8B-Chat-q4_k_m.gguf  
# 默认监听 http://localhost:8080
```

导入 Ollama （请参看 <https://ollama.com/>）

```
# 在 GGUF 文件所在目录创建 Modelfile  
echo "FROM ./Qwen1.5-1.8B-Chat-q4_k_m.gguf" > Modelfile  
  
# 导入模型  
ollama create Qwen1.5-1.8B-Chat-gguf -f Modelfile  
  
# 运行对话  
ollama run Qwen1.5-1.8B-Chat-gguf
```